

Problem V: The Empire Strikes Back

Much like the first problem, this one tests basic skills in interpretation and implementation. The difference here is that neither looping nor if-statements can be avoided. Given two integers N and M , the question asks us to take N integer masses and add up all the ones which are less than or equal to M . If you solved the last problem, chances are you'll have no trouble with looping/if-statements and input. This is as easy as looping N times to read in the masses one by one, and having one if-statement within the loop to check if the number that was just read in is at most M .

Note that we don't have to overcomplicate the solution by storing the numbers into arrays. We can simply process the input one by one as we go, since M is given beforehand and a number won't be needed again after we're done with it for the first time. The only part that you might trip up on is interpreting the phrase "Luke's control of the Force is only strong enough to allow him to lift a rock if its mass is **no larger** than M ." This means that we should add the number if it's *less than or equal to* ($\leq$) M , and not if it's strictly *less than* ($<$) M . Otherwise, the relatively straightforward solution is given below.

Official Solution (C++)

```
#include <iostream>
using namespace std;

int N, M, W, ans = 0;

int main() {
    cin >> N >> M;
    for (int i = 0; i < N; i++) {
        cin >> W;
        if (W <= M) {
            ans += W;
        }
    }
    cout << ans << endl;
    return 0;
}
```

Problem VI: Revenge of the Sith

This problem required arrays and some slightly more complex control structures, but is ultimately nothing too daunting. We are given a distance R , S stormtroopers each associated with a type and position, as well as E Ewoks each associated with a position. First we must first count, for each Ewok, not how many stormtroopers are near him or her, rather how many *types* of stormtroopers are near him or her. Then for each Ewok, if the number of types of stormtroopers is greater than a certain number, then the Ewok is considered in danger and we increment the answer by 1. But what is this number? Once again, your interpretation skills are needed for the phrase "any **more than a single type** of stormtrooper poses a risk to them," which means greater than one (> 1).

Following the KISS principle, the first solution you should consider is to simply do exactly that— for each Ewok, loop and count the number of types of stormtroopers that are in range. Since there are only up to 100 Ewoks and 100 stormtroopers, the total time complexity will be proportional to $O(S \times E)$, which is at most 10000 steps and will certainly pass under the time limit of 2.00 seconds (30 million "steps" per 1 second of time limit is a good rule of

thumb). Although this thought might have not even crossed your mind, you should start looking into the running time analysis of algorithms, which will certainly be important for more advanced problems.

In any case, how should we check if an Ewok is within range of stormtrooper? The formula for Euclidean distance is already given in the question. We just have to see if the value of $\sqrt{(x^2 + y^2)}$ is *less than or equal to* (once again, not *less than*) the value of R , where x is the difference in x -coordinates and y is the difference in y -coordinates. Now, an important thing to point out is that, while we could use the square root function and replicate the distance formula exactly and compare it with R afterwards, this is not a good practice. The square root function always returns a real number (i.e. a floating point value). Floating-point arithmetic is inevitably inaccurate, and directly comparing them with the relational operators will often break down. In the general case, epsilon comparisons provide a fair solution, but there is a popular trick for the distance formula – since both sides of the comparison are guaranteed positive values, we can just get the same result as “ $\sqrt{(x^2 + y^2)} \leq R$ ” by comparing their squares. In other words, we can avoid floating-points altogether with expression “ $x^2 + y^2 \leq R^2$ ”. Though the bounds in this problem may not have been able to expose this bug in many programs, you should start adapting this practice as you improve.

Finally, to keep track of the types, we make a 0-initialized array for each Ewok we input, storing the counts of each type of stormtrooper. This array won't be a burden to the efficiency of the program since there are only up to 4 types, which can be effectively considered a constant. For each of the stormtroopers we loop through and find to be near the current Ewok, we set the count of that type of stormtrooper to 1 (read: *not* increment the count, just set it to 1). Finally, we add up all of the 4 counts to see if the total number is greater than 1. If so, then we increment the answer.

Official Solution (C++)

```
#include <iostream>
using namespace std;

int S, E, R, x, y, ans = 0;
int W[105], X[105], Y[105];

int main() {
    cin >> S >> E >> R;
    for (int i = 0; i < S; i++) {
        cin >> W[i] >> X[i] >> Y[i];
    }
    for (int e = 0; e < E; e++) {
        cin >> x >> y;
        int cnt[5] = {0};
        for (int i = 0; i < S; i++) {
            if ((X[i] - x) * (X[i] - x) + (Y[i] - y) * (Y[i] - y) <= R * R) {
                cnt[W[i]] = 1;
            }
        }
        if (cnt[1] + cnt[2] + cnt[3] + cnt[4] > 1) {
            ans++;
        }
    }
    cout << ans << endl;
    return 0;
}
```